GIT INTERVIEW QUESTIONS & ANSWERS

1. GIT FUNDAMENTALS

1.1 What is Git ?

Answer: Git is a Distributed Version Control System (DVCS) used to track changes in source code during software development. It helps developers to collaborate, maintain history, and work on a project efficiently. It was created by Linus Torvalds in 2005.

Key Points

- Tracks every change in the project.
- Maintains complete history.
- Supports branching and merging.
- Works offline.
- Fast, lightweight and reliable.
- Open source and free.

1.2 What is Version Control System (VCS) ?

Answer: A Version Control System (VCS) is a system that records changes to files over time so that we can track, manage and restore them. It allows multiple developers to work on the same project without overwriting each other's work.

Types of VCS

1. Centralized VCS (CVCS)	2. Distributed VCS (DVCS)
Example: SVN	Example: Git
• One central server stores all versions.	• Every developer has a full copy of the repository.
• Developers check in and check out code.	• Can work offline.

1.3 Git vs GitHub ?

Git	GitHub
• Git is a tool (software).	• GitHub is a platform (service).
• It is used for version control.	• It is used to host Git repositories.
• It works on local machine.	• It provides web interface.
• It is installed on your local system.	• It is cloud based.
• It tracks changes locally.	• It helps in collaboration and code sharing.

→ In simple words: Git is a tool to manage code, GitHub is a platform to store and share that code.

Easy Analogy

Git = Local (Your Laptop)



GitHub

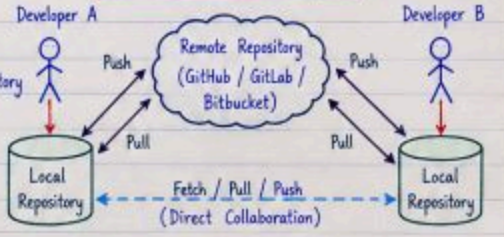
= Remote (Cloud)

1.4 Git Architecture (High Level)

Answer:

Git follows a distributed architecture where each developer has a complete copy of the repository including the full history.

Distributed Architecture of Git



GIT Interview Questions & Answers

2. Git Workflow & Repository

2.1 Git Workflow - The Big Picture

Answer:

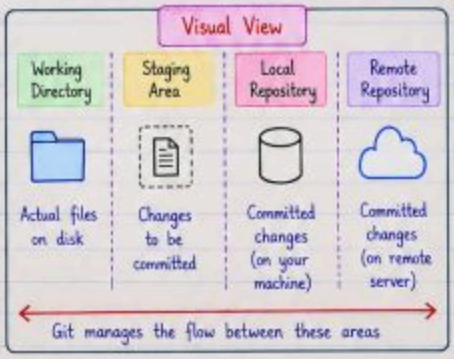
Git follows a simple yet powerful workflow. Changes are made in the working directory, then moved to staging area, committed to local repository, and finally pushed to remote repository.



2.2 Important Areas in Git

Answer:

1. **Working Directory :**
   It is the folder on your local machine where you create, edit and delete files.
2. **Staging Area (Index) :**
   It is a temporary area where you prepare changes before committing.
3. **Local Repository :**
   It is a hidden .git folder where Git stores all the committed changes and metadata.
4. **Remote Repository :**
   It is a version of your project that is hosted on the internet or network.



2.3 Key Commands to Work with Repositories

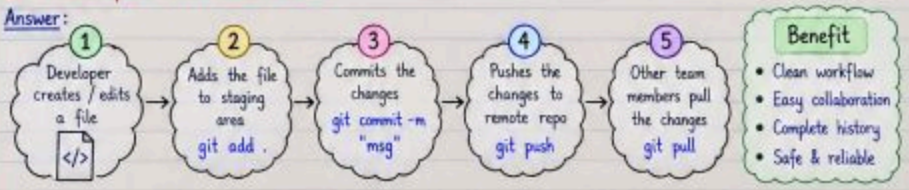
Command	Description
git init	Initializes a new Git repository in the current directory.
git clone <url>	Clones an existing repository from remote to local.
git status	Shows the current status of working directory and staging area.
git add <file>	Adds a file to the staging area.
git add .	Adds all changes in the working directory to staging area.
git commit -m "message"	Commits the staged changes to local repository.
git push origin <branch>	Pushes your committed changes to remote repository.
git pull origin <branch>	Fetches and merges changes from remote repository.

Interview Tip

Always explain Git as a flow of changes from local to remote. Interviewers love candidates who can explain with clear diagrams and real-world examples.

2.4 Example - Real Time Flow

Answer:



Benefit

- Clean workflow
- Easy collaboration
- Complete history
- Safe & reliable

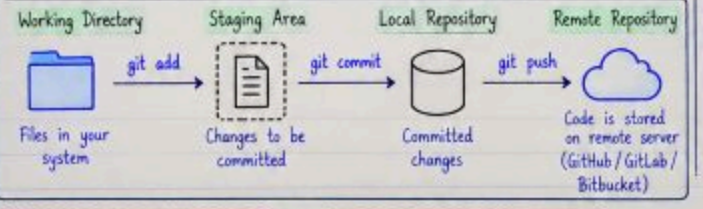
GIT INTERVIEW QUESTIONS & ANSWERS

3. BASIC GIT COMMANDS (PART - 1)

Below are some most commonly used basic Git commands every developer should know.

Command	Description	Example / Notes
1. git add <file>	Adds a specific file to the staging area.	$ git add index.js # stage index.js
2. git add .	Stages all changes (new, modified, deleted) in the current directory.	$ git add . # stage all files
3. git commit -m "msg"	Commits staged changes to the local repository with a message.	$ git commit -m "Initial commit" # creates a new commit
4. git log	Shows the commit history.	$ git log # view comits in reverse branch
5. git diff	Shows the difference between working directory and staging area (unstaged changes).	$ git diff # shows not staged changes
6. git diff --staged	Shows the difference between staging area and last commit (staged changes).	$ git diff --staged # shows staged but not committed
7. git show <commit>	Shows the details of a specific commit.	$ git show a1b2c3d # show commit details

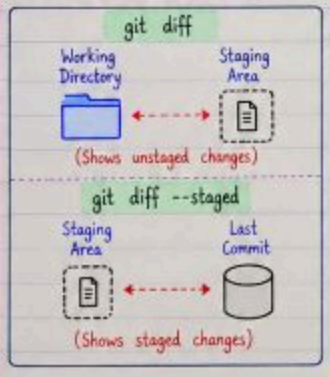
★ How git add, commit and push work together ?



Key Takeaway

Always follow this flow. First stage your changes with git add, then commit them and finally push to remote.

3.1 Understanding git diff



3.2 Other Helpful Commands

Command	Description	Example / Notes
git rm <file>	Removes a file from working directory and stages the deletion.	$ git rm app.js
git mv <old> <new>	Renames or moves a file and stages the change.	$ git mv old.txt new.txt
git status	Shows current status of working directory and staging area.	$ git status
git log --oneline --graph	Shows compact commit history with graph.	$ git log --oneline --graph
git show HEAD	Shows the latest commit details.	$ git show HEAD
git branch	Shows list of branches.	$ git branch
git merge <branch>	Merges a branch into current branch.	$ git merge feature

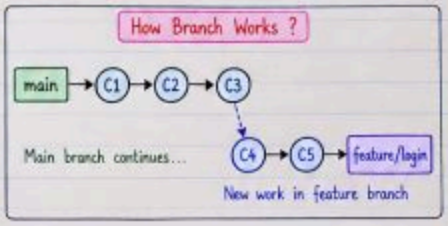
GIT INTERVIEW QUESTIONS & ANSWERS

4. GIT BRANCHING (PART-1)

4.1 What is Branch in Git ?

Answer: A branch in Git is a lightweight movable pointer to a commit. It allows developers to work on new features, bug fixes or experiments without affecting the main codebase.

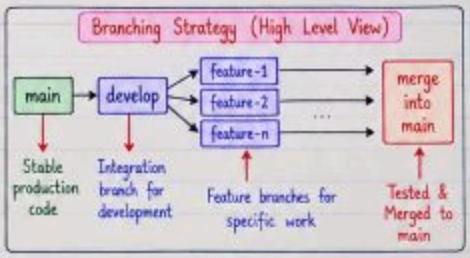
Once the work is complete, changes from the branch can be merged back into the main branch.



4.2 Why Branching is Important ?

Answer:

- Working on multiple features at the same time.
- Isolating new development from stable code.
- Easy collaboration among team members.
- Safe experimentation and quick rollbacks.
- Maintaining a clean and stable main branch.



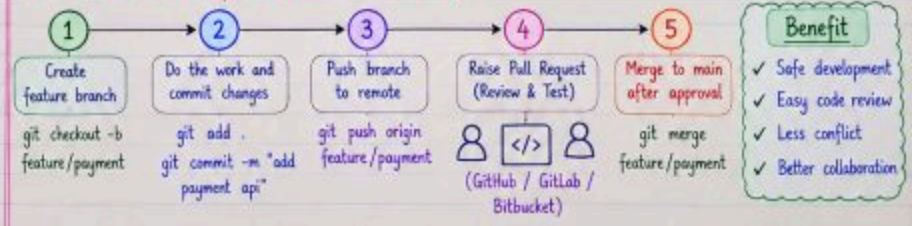
4.3 Basic Branching Commands

Command	Description	Example / Notes
git branch	List all branches in the repository.	git branch # shows local branches
git branch -r	List all remote branches.	git branch -r # shows remote branches
git branch <branch-name>	Create a new branch.	git branch feature/login
git checkout <branch-name>	Switch to a branch.	git checkout feature/login
git switch <branch-name>	Switch to a branch (modern command).	git switch feature/login
git checkout -b <branch-name>	Create and switch to a new branch.	git checkout -b feature/signup
git branch -d <branch-name>	Delete a branch (safe delete).	git branch -d feature/login
git branch -D <branch-name>	Force delete a branch.	git branch -D feature/login
git branch -m <new-name>	Rename current branch.	git branch -m feature/auth
git branch -M <new-name>	Force rename branch.	git branch -M new-feature

Interview Tip

When explaining branching in interview, always show the diagram and use real examples from your experience. It makes your answer more impactful.

4.4 Real Time Example - Feature Branch Workflow

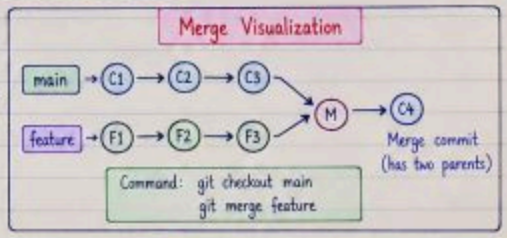


GIT Interview Questions & Answers

5. Merge vs Rebase

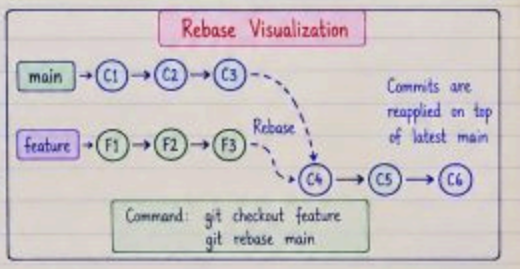
5.1 What is Merge ?

Answer: Merge is the process of combining changes from one branch into another. It creates a new merge commit that has two parents (one from the target branch and one from the source branch). It preserves the complete history of both branches.



5.2 What is Rebase ?

Answer: Rebase is the process of moving or replaying a sequence of commits to a new base commit. It reapplies your changes on top of the latest commit of the target branch. It does not create a merge commit and results in a linear history.



5.3 Types of Merge

Type	How it happens	Diagram	Description
1. Fast Forward Merge	When target branch has no new commits, target just moves to source branch.	C1 → C2 → C3 → C4	No new merge commit is created. Target branch simply moves forward.
2. Three-way Merge	When both branches have changes.	C1 → C2 → M; F1 → F2 → M	Creates a new merge commit (M) with two parents.
3. Squash Merge	When we want to merge all commits as a single commit (No history).	C1 → C2 → S; F1 → F2 → F3 → S	All feature branch commits are squashed into one single commit (S).
4. No Fast Forward Merge	When we disable fast forward merge (--no-ff).	C1 → C2 → M; F1 → F2 → M	Creates a merge commit (M) even when fast forward was possible.

5.4 Merge vs Rebase – Key Differences

Aspect	Merge	Rebase
History	Creates a merge commit. History looks like a graph.	Creates a linear history.
Commit IDs	Original commit IDs are preserved.	New commit IDs are created.
Use Case	When working on shared branches (team collaboration).	When keeping a clean and linear history.
Safety	Safe, doesn't rewrite history.	Rewrites history, so it can be risky.
Conflicts	Can have conflicts.	Can have conflicts (during reapplying commits).
Command	git merge <branch-name>	git rebase <branch-name>

5.5 When to Use ?

- Use Merge for long lived branches like develop, release, main.
- Use Rebase for short lived feature branches (before merging).
- Use Squash Merge when you want a single clean commit.
- Always test your code after merge or rebase.

☆ Interview Tip

- Always explain with diagrams.
- Mention real world use cases from your experience.
- Talk about advantages and disadvantages.
- Clearly explain when to use merge and when to use rebase.

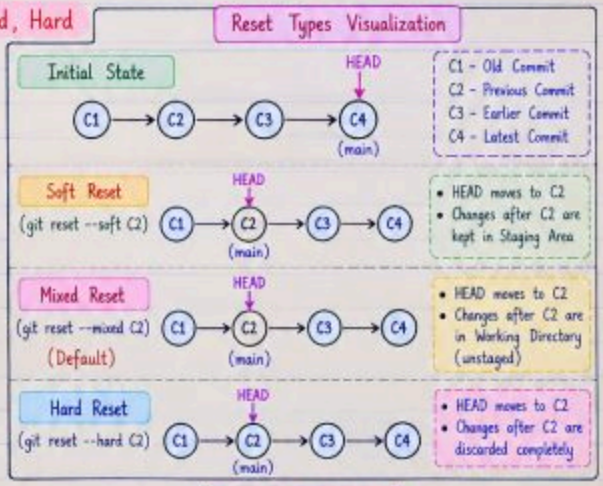
GIT INTERVIEW QUESTIONS & ANSWERS

6. UNDO OPERATIONS IN GIT

6.1 Git Reset - Soft, Mixed, Hard

Answer:

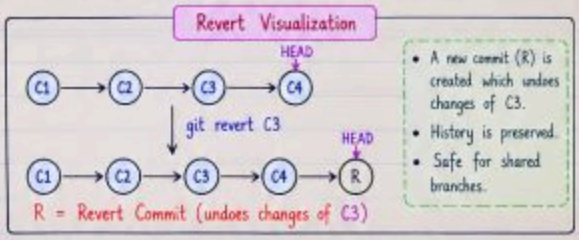
git reset is used to move the HEAD and current branch pointer to a specific commit.
It changes what is staged in the index and what is in your working directory.



6.2 Git Revert

Answer:

git revert is used to create a new commit that undoes the changes introduced by a specific commit.
It is safe and does not rewrite history.



6.3 Git Restore (Newer Command)

Answer:

git restore is used to discard changes in working directory or unstage files from staging area.

Use Case	Command	Description
Unstage a file	git restore --staged <file>	Removes file from staging area (keeps changes in working directory).
Discard changes in working directory	git restore <file>	Discards changes in working directory (reverts to last commit).
Discard changes in working directory (including untracked)	git restore -s HEAD <file>	Same as above (explicitly restore from HEAD).

6.4 Summary - Reset vs Revert vs Restore

Feature	Reset	Revert	Restore
Purpose	Move HEAD to a specific commit.	Undo changes by creating a new commit.	Discard changes in working directory or staging area.
History Rewrite	Yes (Soft/Mixed/Hard)	No	No
Safe for Shared Branches	Hard Reset - No Soft/Mixed - Use Carefully	Yes (Always Safe)	Yes
Affects Staging Area	Yes	No	Yes / No (Depends on command)
Commits Lost	Hard Reset - Yes Soft/Mixed - No	No	No
Typical Use	When you want to move back or clean history (local).	When you want to undo changes without affecting history.	When you want to discard local changes or unstage files.

GIT Interview Questions & Answers

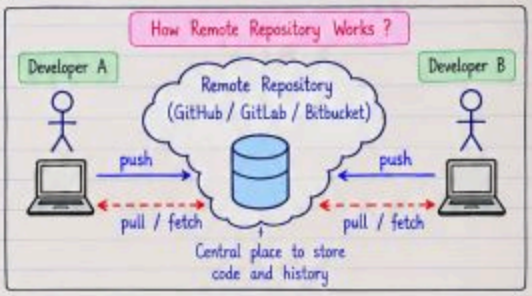
7. Remote Repository & Collaboration

7.1 Remote Repository - Overview

Answer :

Remote repository is a version of your project that is hosted on the internet or network. It allows multiple developers to collaborate, share code and maintain a central source of truth.

Examples : GitHub, GitLab, Bitbucket.



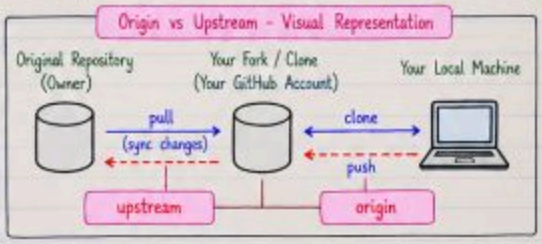
7.2 Important Remote Commands

Command	Description	Example	Notes
git remote -v	List all remote repositories.	git remote -v	Shows fetch and push URLs.
git remote add <name> <url>	Add a new remote repository.	git remote add origin https://github.com/user/repo.git	'origin' is default name.
git fetch <remote>	Downloads changes from remote but does not merge.	git fetch origin	Safe - does not change working directory.
git pull <remote> <branch>	Fetch and merge changes from remote branch.	git pull origin main	Shortcut for fetch + merge.
git push <remote> <branch>	Push local commits to remote branch.	git push origin main	Requires permissions.
git clone <url>	Clone a remote repository locally.	git clone https://github.com/user/repo.git	Creates local copy.
git remote remove <name>	Remove a remote repository.	git remote remove origin	Deletes remote alias.

7.3 Origin vs Upstream

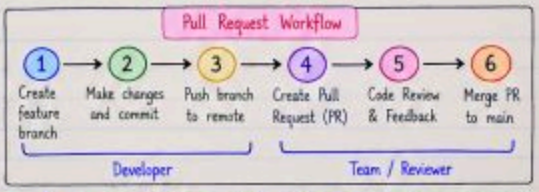
Origin :
- Refers to your fork or your personal remote repository. It is where you push your changes.
- Example : origin

Upstream
- Refers to the original repository from which you forked or cloned. It is usually read-only for contribution.
- Example : upstream



7.4 Pull Request (PR) - Basics

Answer : A Pull Request (PR) is a request to merge your changes from a feature branch into the main branch of a repository. It allows code review, discussions and testing before merging.



Benefits :
- Code review before merging.
- Better collaboration and communication.
- Helps maintain code quality.

Key Points
- Pull changes from upstream.
- Push changes to origin.
- Use Pull Request for collaboration.
- Always explain the purpose of each command with real-world examples.

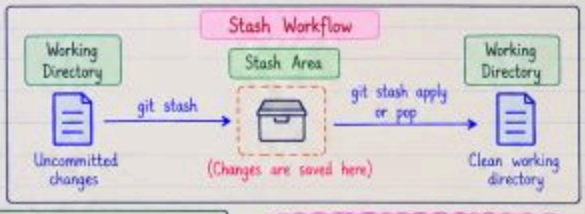
GIT INTERVIEW QUESTIONS & ANSWERS

8. ADVANCED GIT TOPICS

8.1 Git Stash

Answer :

Git stash is used to save your uncommitted changes temporarily and clean your working directory so you can work on something else.



Important Stash Commands

Command	Description
git stash	Stash current changes.
git stash list	List all stashes.
git stash apply	Apply the latest stash without deleting it.
git stash pop	Apply the latest stash and remove it.
git stash drop	Delete a specific stash.
git stash clear	Delete all stashes.

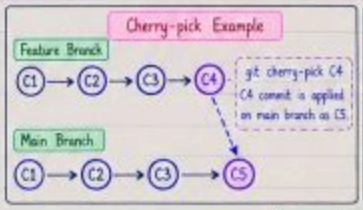
Interview Tip

- Stash is very useful when you are working on something and need to switch branches quickly.
- It helps to keep your working directory clean without losing changes.

8.2 Git Cherry-pick

Answer :

Git cherry-pick is used to apply a specific commit from one branch to another branch.



Cherry-pick Commands

Command	Description
git cherry-pick <commit-id>	Apply specific commit.
git cherry-pick <commit-id1> <commit-id2>	Apply multiple commits.
git cherry-pick --abort	Abort cherry-pick.
git cherry-pick --continue	Continue after resolving conflict.

8.3 Git Tags

Answer :

Tags are used to mark a specific point in history which is usually used for releases.

Types of Tags

- Lightweight Tag – simple pointer to a commit.
- Annotated Tag – contains metadata like tag message, author, date etc.

Tag Commands

Command	Description
git tag	List all tags.
git tag <tag-name>	Create lightweight tag.
git tag -a <tag-name> -m "msg"	Create annotated tag.
git show <tag-name>	Show tag details.
git push origin <tag-name>	Push tag to remote.
git push origin --tags	Push all tags.

8.4 Git Ignore

Answer :

.gitignore file is used to tell Git which files or directories to ignore so that they are not tracked in the repository.

Example .gitignore content

```
# Ignore log files
*.log
# Ignore node modules
node_modules/
# Ignore build files
dist/
build/
# Ignore environment files
.env
```

Key Point

- .gitignore is very important to keep your repository clean.
- It helps to avoid pushing unnecessary files.
- Commonly used for log files, build files, dependencies and environment files.

8.5 Git Hooks (Introduction)

Answer :

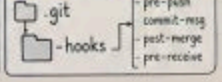
Git hooks are scripts that Git runs automatically before or after certain events like commit, push, merge etc.

Common Hooks

Hook	Purpose
pre-commit	Run before commit. Used for code formatting, linting.
commit-msg	Validate commit message.
pre-push	Run before pushing to remote.
post-merge	Run after merge.
pre-receive	Run on server side before receiving push.

Hooks Location

Hooks are stored in .git/hooks/ directory of your repository.



GIT Interview Questions & Answers

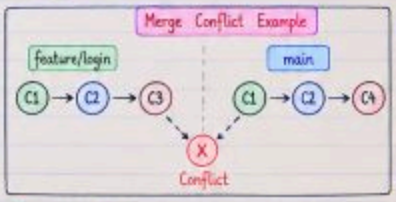
9. Real-Time Scenarios (with Solutions)

In real projects, we face many situations where we need to fix mistakes, recover data or resolve conflicts. Below are important scenarios that are frequently asked in interviews.

9.1 Merge Conflict - How to Resolve ?

Answer :

1. Git shows conflict in the file.
2. Open the file and look for conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`).
3. Edit the file and keep the correct code.
4. `git add <file>`
5. `git commit`



Key Tip

- Always pull latest changes before working to avoid conflicts.
- Communicate with team members.

9.2 Accidentally Commited to Main Branch

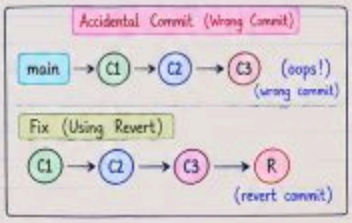
Answer :

If commit is not pushed:

1. `git reset --soft HEAD~1` (undo last commit)
2. `git checkout -b feature-branch`
3. `git cherry-pick <commit-id>`

If commit is already pushed:

1. `git revert <commit-id>` (safe option)
2. Push the revert commit.



When to Use ?

- Use revert if commit is already pushed (shared branch).
- Use reset + cherry-pick if commit is not pushed.

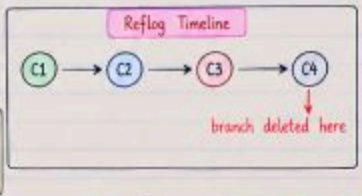
9.3 Deleted Branch - How to Recover ?

Answer :

1. `git branch -a` (list all branches)
2. Find the deleted branch in reflog.
3. `git checkout -b <branch-name> <commit-id>`

Example (from reflog output)

```
a1b2c3d (HEAD -> main)  HEAD@{0}: commit: fix bug
d4e5f6g HEAD@{1}: checkout: moving from feature to main
h7i8j9k HEAD@{2}: branch: deleted branch feature/login
```



Interview Tip

- Reflog is very powerful. It can save you in most of the situations.
- Use reflog to find the lost commits, branches or changes.

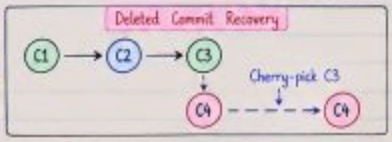
9.4 Accidentally Deleted Last Commit

Answer :

1. `git reflog`
2. Copy commit-id of the deleted commit.
3. `git cherry-pick <commit-id>`

Key Point

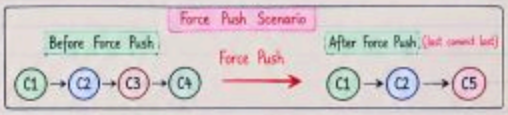
- Use reflog to find the commit and cherry-pick to bring it back safely.



9.5 Force Push Gone Wrong

Answer :

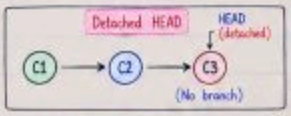
1. Check what was pushed : `git reflog`
2. Find the correct commit-id.
3. `git push origin <branch-name> <commit-id> --force` (Use `--force-with-lease` for safety)



9.6 Detached HEAD - What & How to Fix ?

Answer :

- Detached HEAD means you are not on any branch.
- You are pointing to a commit directly.
- Solution: Create a new branch from that point.



Fix :

1. `git checkout -b new-branch`
2. Now HEAD is on the branch.

9.7 Stash Important Work

Answer :

1. `git stash` (save changes)
2. `git stash list` (view stash)
3. `git stash pop` (apply & remove)
4. `git stash apply` (apply but keep)
5. `git stash drop` (delete specific)
6. `git stash clear` (delete all)



Golden Rule

- Keep your working directory clean.
- Use stash when you need to switch branches quickly.
- Helps to avoid unnecessary commits.

GIT Interview Questions & Answers

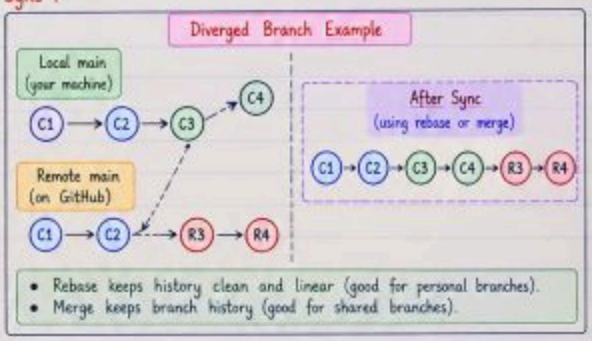
9. Real-Time Scenarios (with Solutions)

9.8 Diverged Branches - How to Sync ?

Answer :

When your local and remote branch have different commits, you need to sync them.

1. git fetch origin (get latest changes)
2. git log HEAD..origin/main (see what's missing)
3. git pull --rebase origin main or git merge origin/main (sync changes)
4. Resolve conflicts (if any)
5. git push origin main

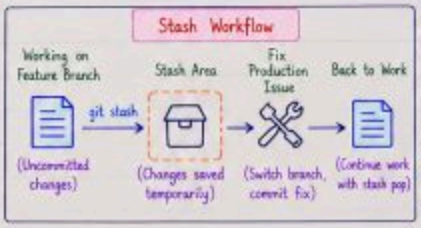


9.9 Stash Use Case in Real Life

Answer :

You are working on a feature but suddenly need to fix a production bug on another branch.

1. git stash (save your work)
2. git checkout -b bug-fix
3. Fix and commit the bug
4. Switch back to feature branch
5. git stash pop (bring back your work)



Interview Tip

- Stash is very useful when you need to switch branches quickly without losing your current work.

9.10 Submodules - When & How to Use ?

Answer :

Submodules allow you to include another Git repository inside your repository.

Use Case: When you want to reuse a library or component maintained in a separate repository.

Common Submodule Commands	Description
git submodule add <repo-url> <path>	Add a submodule.
git submodule update --init --recursive	Initialize submodules.
git submodule foreach git pull origin main	Update all submodules.
git submodule deinit -f <path>	Remove submodule.

9.11 Best Practices & Tips

1. Commit Messages

- Be concise and clear.
- Use imperative mood.
- Explain "why", not just "what".
- Example: Fix login issue on checkout page

2. Small Commits

- Make small, atomic commits.
- Easy to review & revert.
- One logical change per commit.

3. Branching Strategy

- Use feature branches.
- Keep main branch clean.
- Follow team strategy (Gitflow / Trunk Based).
- Delete merged branches.

4. Code Review

- Review before merge.
- Follow checklists.
- Give constructive feedback.
- Discuss and test changes.

5. Clean History

- Rebase before merge.
- Squash fixup commits.
- Remove unnecessary WIP commits.
- Keep a meaningful commit history.

9.12 Frequently Asked Interview Questions (Quick List)

1. What is Git and why is it used ?
2. What is the difference between Git and GitHub ?
3. Explain Git architecture.
4. What is the difference between merge and rebase ?
5. What is the purpose of git stash ?
6. How do you resolve merge conflicts ?
7. How will you recover a deleted branch ?
8. What is a pull request ?
9. How will you handle merge conflicts ?
10. What is the use of .gitignore ?
11. What are Git hooks ?
12. What is a submodule and when to use it ?

Golden Rule

- Understand the concept, practice in real scenarios and keep learning.
- Use Git in your daily projects.
- Clean, meaningful commits make you a better developer.

GIT INTERVIEW SCENARIOS & ANSWERS (Part 1)

Q1. You accidentally committed directly to the main branch. What will you do ?

Answer :

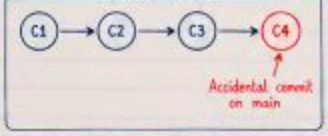
This is a very common production scenario. Direct commits to the main branch should be avoided because the main branch contains production-ready code. If you have already committed directly, we can move the commit to a feature branch or revert it.

A. If commit is NOT PUSHED to remote (Move to new branch)

1. Create a new branch from the current commit.
2. Reset main to remove the commit.
3. Switch to the new branch.
4. Continue your work.

```
git reset --soft HEAD~1              # Undo the last commit but keep changes staged
git checkout -b feature/payment-fix  # Create proper feature branch
git commit -m "Payment API Fix"      # Commit on feature branch
git push origin feature/payment-fix  # Push feature branch
```

Scenario Flow



B. If commit is ALREADY PUSHED to remote (Use Revert)

1. Create a new commit that undoes the changes.
2. This is safe for shared branches and keeps history clean.
3. Push the revert commit.

```
git revert <commit-id>       # Creates a new commit that undoes changes
git push origin main         # Push revert commit
```

Interview Tip :

- Never use git reset --hard on a shared or production branch.
- Always prefer revert or move the commit to a feature branch to avoid rewriting history.

Q2. A merge conflict occurs while merging your feature branch. How will you resolve it ?

Answer :

A merge conflict occurs when two developers modify the same part of the same file. Git cannot automatically decide which change to keep, so we need to resolve the conflict manually.

Conflict Example (file.txt)

```
<<<<<<< HEAD
Current Code
=======
Incoming Code
>>>>>>> feature/login
```

Resolution Steps

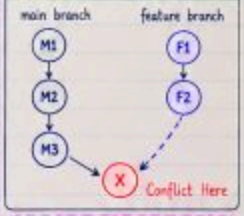
1. Pull the latest changes from main and try merging.

```
git pull origin main          # Get latest changes
git merge feature/login       # Merge your feature branch
```

2. Git will show conflict markers in the file.
3. Open the conflicted file.
4. Compare both changes and keep the correct code.
5. Remove the conflict markers (<<<<<<<, =======, >>>>>>>).
6. Save the file.
7. Add resolved files, commit and push.

```
git add .
git commit -m "Resolved merge conflict"
git push origin main
```

Merge Conflict Scenario



Interview Tip :

- Always communicate with the other developer before resolving a conflict.
- Understand the code changes and keep the correct logic.

Q3. You deleted a Git branch by mistake. Can you recover it ?

Answer :

Yes, we can recover a deleted branch using Git Reflog. Reflog records every action done in your repository (checkout, commit, branch delete, reset, etc.).

Example (Reflog Output)

```
$ git branch
* main
  feature-payment    (deleted)
$ git reflog
a7c56d (HEAD -> main) HEAD@{0}:
  checkout: moving from feature-payment to main
b34f98 HEAD@{1}: commit: payment integration
1a2b3c HEAD@{2}: checkout: moving from main to
  feature-payment
```

Recovery Steps

1. Run git reflog and find the commit-id where the branch existed.

```
git reflog
```

2. Create the branch again using that commit-id.

```
git checkout -b feature-payment a7c56d
# Create branch from the commit
```

3. Push the branch to remote (if needed).

```
git push origin feature-payment
```

Visual Representation



Key Point

- Use git reflog to find lost commits or branches.
- Even if a branch is deleted, commits are not immediately lost.
- Create a new branch from the commit-id and push it to remote.

GIT Interview Scenarios & Answers (Part 2)

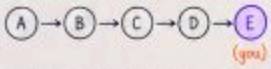
Q4. One developer force pushed the repository and your commits disappeared. What will you do ?

Answer :

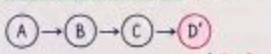
Force push rewrites the history on the remote which can remove commits from other developers' local repositories. We can recover the lost commits using Git Reflog.

What happened ?

Before Force Push (your commits)



After Someone Force Pushed (others)



Interview Tip :

- Always prefer git push --force-with-lease to avoid overwriting teammates' work accidentally.

Recovery Steps

1. Run git reflog to see all your previous actions.

```
git reflog
```

2. Find the commit-id of your lost commit.

```
git reflog
```

```
e1a2b3c HEAD@{0}: commit: Update README
d4f5g6h HEAD@{1}: commit: Add feature
c7d8e9f HEAD@{2}: commit: Payment integration
....
```

3. Create a new branch from that commit.

```
git checkout -b recovery-branch <commit-id>
```

4. Cherry-pick or merge your commits to the current branch.

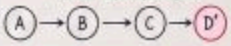
```
git cherry-pick <commit-id>
```

3. Resolve conflicts (if any), commit and push.

```
git add ..
git commit -m "Recovered lost commit"
git push origin recovery-branch
```

Recovery Visual

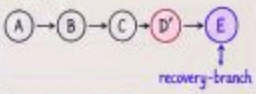
Remote (after force push)



Your reflog (shows old commits)

```
e1a2b3c  commit: E (HEAD@{2})
d4f5g6h  commit: D (HEAD@{3})
c7d8e9f  commit: C (HEAD@{4})
.....
```

Recovered



Best Practice :

- Never force push on main branch.
- Communicate before any force push.
- Use protected branches in GitHub/ GitLab.
- Take regular backups (reflog helps).

Q5. You have uncommitted changes but suddenly need to fix a production issue. What will you do ?

Answer :

Use Git Stash to temporarily save your work, switch to the required branch, fix the issue and then come back to your work.

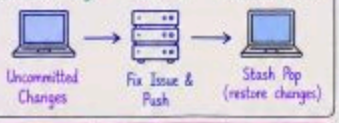
Why use Stash ?

- It saves your uncommitted changes safely.
- Your working directory becomes clean.
- You can focus on the urgent task without losing your work.

Interview Tip :

- Stash is very useful in real production environment.
- Use descriptive stash messages.
- git stash save "WIP: payment integration"

Scenario Flow



Example (Commands)

```
# 1. Check current changes
$ git status
# 2. Save changes to stash
$ git stash          # or
$ git stash save "WIP: payment integration"
# 3. Switch to production branch
$ git checkout production
# 4. Fix the issue and commit
$ git add ..
$ git commit -m "Fix production issue"
$ git push origin production
# 5. Switch back to your feature branch
$ git checkout feature/payment
# 6. Bring back your stashed work
$ git stash pop      # restores and removes stash
# If multiple stashes exist:
$ git stash list
$ git stash apply stash@{n}  # keep stash
$ git stash pop stash@{n}    # apply and remove
```

Stash Workflow (Step by Step)

1. Save your work using stash.

```
git stash          # Saves changes
```

2. Switch to production branch.

```
git checkout production
```

3. Fix the issue and commit.

```
git add ..
git commit -m "Fix production issue"
git push origin production
```

4. Come back to your branch.

```
git checkout feature/payment
```

5. Bring back your work.

```
git stash pop      # Restore your changes
```

General Tips Interviewers Expect

- ✔ Understand the root cause before applying any command.
- ✔ Keep main branch clean and stable.
- ✔ Use meaningful commit messages.
- ✔ Prefer small and frequent commits.
- ✔ Be comfortable with stash, reflog and branch operations.
- ✔ Communicate with your team in real scenarios.